

Course Introduction & Basic Syntax

Pablo E. Gutiérrez-Fonseca, PhD

BIOL-4994 & BIOL-4991 | BIOL-6994 & BIOL-6997
Fall 2025
2025-Aug-01 (updated: 2025-Aug-11)



What is R Programming?

- R is a free and open source software environment and programming language for statistics.
- R was created by Ross Ihaka and Robert Gentleman at the University of Auckland (in New Zealand), and is based on the S language that was created by John Chambers at Bell Laboratories.



Robert Gentleman



Ross Ihaka

What is R Programming?

- When you download and install R, you get a collection of basic packages (and libraries) that can be used to implement several common data manipulations, graphical displays, and statistical models.



Strengths and Weaknesses

- **Strengths:**
 - Free and Open Source.
 - Strong User Community.
 - Highly extensible, flexible.
 - Implementation of high-end statistical methods.
 - Flexible graphics and intelligent defaults.
- **Weakness:**
 - Steep learning curve.
 - Slow for large datasets.

Why R?

- **Computing power:** R can handle much larger datasets than traditional programs, which is especially important for long-term ecological data, community science data, and public health data.
- **Flexibility & convenience:** Traditionally, scientists would need separate programs for data cleaning, statistical procedures, and data visualization. Learning R means you can do everything in a single program, and customize to fit the goals of your project.
- **Reproducibility:** Saving your code as an R script makes it easy for other scientists to see what you did, and repeat your methods.

R Overview



How to download?

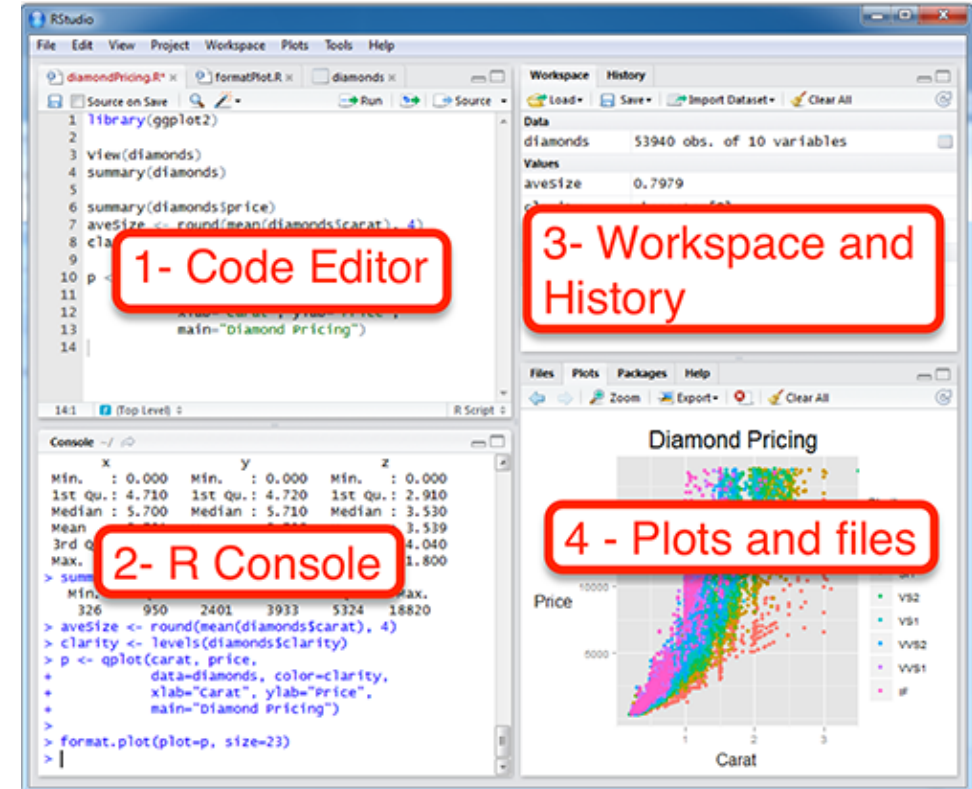
- Google it using R or CRAN (Comprehensive R Archive Network)
 - <http://www.r-project.org>

How to download?

- Having installed R, the next thing we will want to do is install **RStudio**, a popular and useful interface for writing scripts and using R.
- If you google **RStudio** you will get to this window:
<https://www.rstudio.com/products/rstudio/download/>
- Rstudio is an ***integrated development environment (IDE)*** for R that allows users to interact more easily with R by integrating different aspects of scripting, from code completion to debugging.

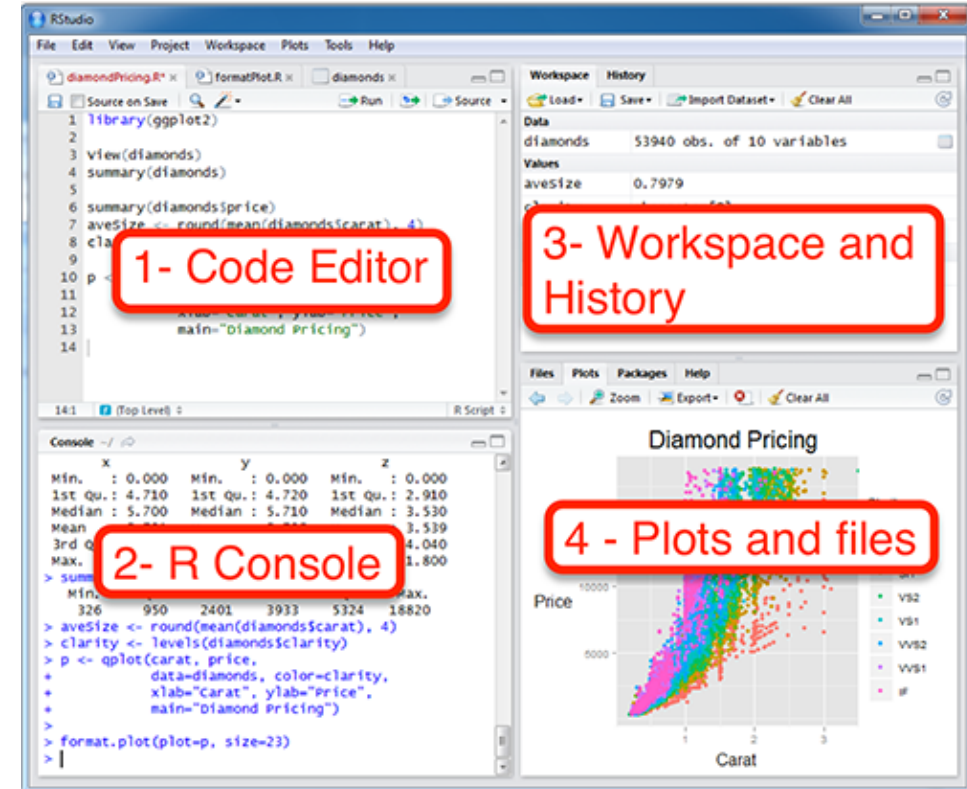
Typical Rstudio session

- **Code Editor:** Contains code to tell R what to do.
- Save scripts for future use.



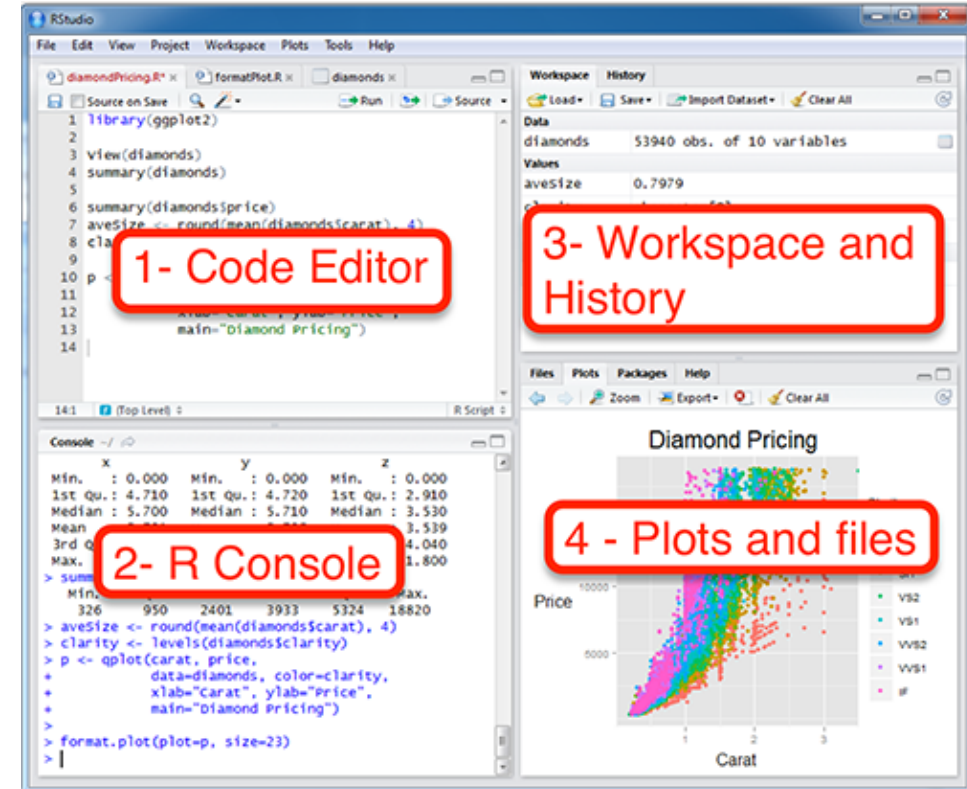
Typical Rstudio session

- **Console:** output & temporary input - usually unsaved.



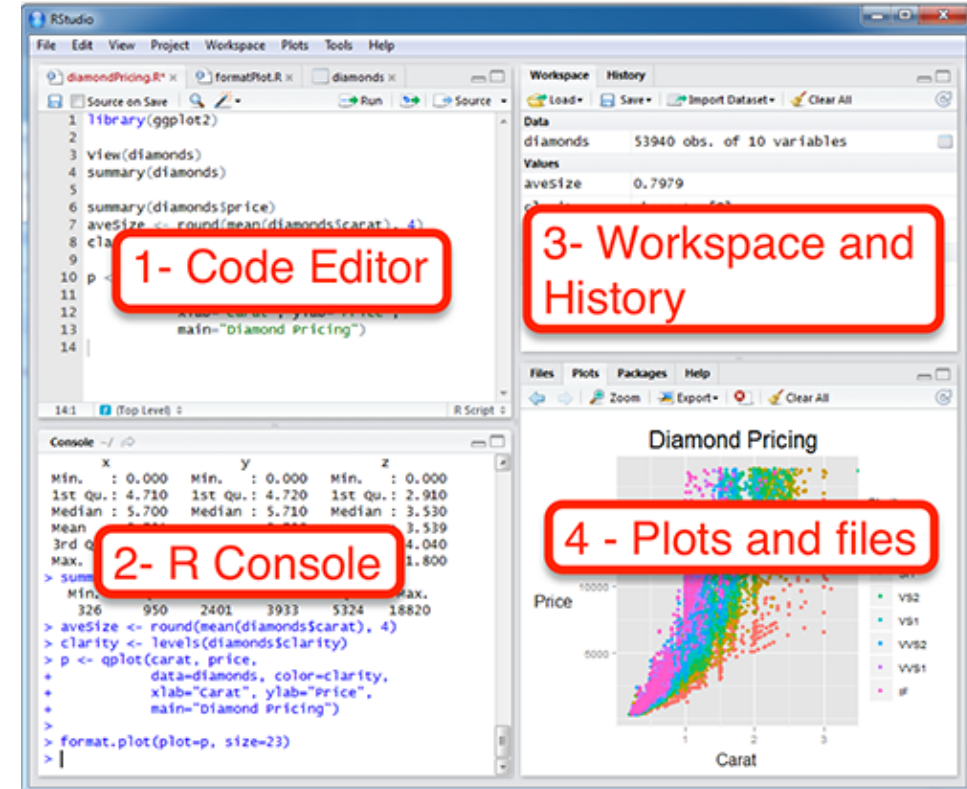
Typical Rstudio session

- **Workspace:** Stores objects created during the session. Save with `save.image()`.
- **History:** Records commands used. Save with `savehistory()`.



Typical Rstudio session

- **Plots:** Displays graphs. Save plots with `ggsave()`.
- **Files:** Shows files in the working directory.



Let's Get to Work!



Let's Get to Work!

- Now that you know how RStudio is organized and how to talk to R...
- It's time to start coding:
 - Create objects
 - Write scripts
 - Explore functions
 - View results in the console and plots panel

Working Console window

You can type commands directly into the **Console**, but they will be forgotten when you close the session.

```
3 + 4
```

```
## [1] 7
```

Script editor

- Use the **Script Editor** to write and save your R code.
 - Lets you keep a record of your work
 - Use `#` to add comments and explanations
 - To run code from the script editor:
 - Click the **Run** button (top right of the script editor), or
 - Use the shortcut Ctrl + Enter (Windows), or
 - Use the shortcut Cmd + Enter (Mac)
 - Ideal for reproducible workflows

```
# Intro to R Lesson
# Feb 16th, 2016

# Interacting with R

# This is a comment explaining the next line
# https://hbctraining.github.io/Intro-to-R/
3 + 4
```

```
## [1] 7
```


Console command prompt

- Interpreting the command prompt can help understand when R is ready to accept commands. Below lists the different states of the command prompt and how you can exit a command:
 - The `>` prompt means R is ready to accept a new command.
 - The `+` prompt means R is waiting for more input (e.g., a missing parenthesis or quote).
 - To cancel an incomplete command, press the `Esc` key.

Exercise

- Highlight only `3 +` in your script editor and run it.
 - What happens in the console?

Exercise

- Highlight only `3 +` in your script editor and run it.
 - What happens in the console?
 - Notice the `+` prompt; R is waiting for more input.
 - Now, bring back the prompt `>` by:
 - Clicking inside the **Console**
 - Pressing the `Esc` key
 - This cancels the incomplete command and resets the console.

Syntax of R program

- Now that you know how to communicate with R through the script editor or console, let's explore how R syntax works.

Syntax of R program

- Below is an example script that highlights key parts of R's "grammar":
 1. `#` This is a comment – used to explain what your code does
 2. Create a variable using the assignment operator `<-`
 - `x <- 5`
 3. Define a function with arguments

```
greet <- function(name) {  
  message <- paste("Hello,", name) # paste  
  return(message)  
}  
  
# Call the function using = for the argument  
greet(name = "Pablo")
```

```
## [1] "Hello, Pablo"
```

Syntax of R program

- Below is an example script that highlights key parts of R's "grammar":

1. `#` This is a comment – used to explain what your code does
2. Create a variable using the assignment operator `<-`
 - `x <- 5`
3. Define a function with arguments

```
greet <- function(name) {  
  message <- paste("Hello, ", name)  # paste  
  return(message)  
}  
  
# Call the function using = for the argument  
greet(name = "Pablo")
```

```
## [1] "Hello, Pablo"
```

Key elements to notice:

1. `#` starts a comment, ignored by R, but helpful for you and others.
2. `<-` is the assignment operator, used to store values or functions.
3. `function()` creates functions you can reuse.
4. `=` is used to assign values to arguments within functions.

`<-` vs `=` in R

`<-` for Assignment Use to assign values to variables

Preferred and standard in R

Example:

```
x <- 10
```

`=` for Function Arguments Use inside functions to assign values to arguments

Example:

```
mean(x = c(1, 2, 3))
```

```
## [1] 2
```

Data Types

- Variables can contain values of specific types within R. The six data types that R uses include:
 1. `numeric` for any numerical value
 2. `integer` for integer numbers (e.g., `2L`, the `L` indicates to R that it's an integer)
 3. `logical` for `TRUE` and `FALSE` (the Boolean data type)
 4. `complex` to represent complex numbers with real and imaginary parts (e.g., `1+4i`) and that's all we're going to say about them

Data structures



Data structures

- R is an object oriented programming language where an **object** is a generic term to describe something in R.
- There are many types of R-objects.
 1. Vectors
 2. Factors
 3. Lists
 4. Matrices
 5. Data Frames

Vectors in R

- Vector is a sequence of elements of the same type (numeric, character, logical).

```
a <- c(1,2,3,4,5)
b <- c(6,7,8,9,10)
```

- Command `c` creates a vector that is assigned to object **a** and **b**

Vectors in R

- Two vectors of the same length can be added ($a+b$), multiply ($a*b$)

```
c <- (a+b)  
c
```

```
## [1]  7  9 11 13 15
```

```
d <- (a*b)  
d
```

```
## [1]  6 14 24 36 50
```

Vectors in R

- Element in a vector can be sorted (*sort()*)

```
e <- c(3,2,20,25,5)  
sort(e)
```

```
## [1] 2 3 5 20 25
```

Factors in R

- Categorical data structure with predefined levels.

```
f <- factor(c("low", 'medium', "high", 'very high'))
```

Data

- R has three general classes for data:
 1. **list**: collection of objects of different lengths or classes
 2. **matrix**: vectors of the same length and same class
 3. **data.frame**: vectors of the same length and different classes

Lists

- A list is an R-object which can contain many different types of elements inside it like vectors, functions and even another list inside it.

```
# Creating a list in R with various types of elements  
my_list <- list(  
  Numbers = c(1, 2, 3, 4, 5), # A numeric vector  
  Words = c("apple", "banana", "cherry")) # A character vector
```

```
my_list
```

```
## $Numbers  
## [1] 1 2 3 4 5  
##  
## $Words  
## [1] "apple" "banana" "cherry"
```


Matrices

- A matrix is a two-dimensional rectangular data set.
- It can be created using a vector input to the matrix function.

```
# Create a 3x3 matrix in R
my_matrix <- matrix(
  data = 1:9,          # Data to fill the matrix
  nrow = 3,            # Number of rows
  ncol = 3,            # Number of columns
  byrow = TRUE)        # Fill the matrix by rows
my_matrix
```

```
##      [,1] [,2] [,3]
## [1,]    1    2    3
## [2,]    4    5    6
## [3,]    7    8    9
```

Data Frames

- Data frames are tabular data objects.
- **Unlike a matrix in data frame each column can contain different modes of data.**
- The first column can be numeric while the second column can be character and third column - Can be logical.
- It is a list of vectors of equal length.
- Data Frames are created using the `data.frame()` function.

Summary

- This table summarizes the main data structures in R by their shape (linear vs rectangular) and type consistency (same vs mixed types).

	Linear	Rectangular	Description / Example
All same type	Vector <code>c(1, 2, 3)</code>	Matrix <code>matrix(1:6, nrow = 2)</code>	Homogeneous values; good for numeric computations and math
Mixed types	List <code>list("a", 1, TRUE)</code>	Data Frame <code>data.frame(name = c("A", "B"), age = c(30, 25))</code>	Heterogeneous columns; ideal for real-world datasets

Reading data



Reading data

- R can read data from various file formats, including:
 - CSV files
 - Text files
 - Excel files
 - other formats like JSON, XML, etc.

Reading data

- **CSV File**

- Usage: Reads comma-separated values, often used for structured data.

```
data <- read.csv("file.csv")
```

- **Text File**

- Usage: Reads tabular data from a text file; sep can be adjusted for different delimiters.

```
data <- read.table("file.txt", header = TRUE, sep = "\t")
```

- **Excel File**

- Usage: Reads data from Excel files; you can specify the sheet number or name.

```
library(readxl)  
data <- read_excel("file.xlsx", sheet = 1)
```

Reading data

- Print the current working directory

```
getwd()
```

- Change the working directory:
 - Replace the path with your actual folder location
 - Use forward slashes /, not backslashes \

```
setwd(mydirectory)  
setwd("c:/docs/mydir")
```

- Pro Tip: The easiest way to set the working directory
 - Go to the RStudio menu:
 - Session > Set Working Directory > Choose Directory...

Reading data

- However, using Session → Set Working Directory or setwd() is mostly for beginners.
- When you create a project (next class), the [here](#) package can be very useful.

```
install.packages('here')  
library(here)  
here()  
  
here("data", "file-name.csv")
```


Reading data

- Subscripts are given inside square brackets after the object's name:
- `df[,1]`
 - Gets the **first column** from the object `dat`
- `df[1,]`
 - Gets the **first row** from the object `dat`
- `df[1,1]`
 - Gets the **first row** and its **first column** from the object `dat`

Assigning Values in R

- In R, you can assign values to objects using the syntax `object <- value`
- An arrow (`<-`) formed by a smaller than character and a hyphen without a space!
- The equal character (`=`).

R Warning!

- Naming objects:
 - R is a case sensitive language.
 - FOO, Foo, and foo are three different objects
 - Object names can't start with a number
 - Never use special characters, such as `Ã`, `Ã¸`, or `Ã` in object names.

Using Functions in R

- R is a function based language where a ***function*** takes in some input and produces some output.
 - Vegas rules: what happens in a function, stays in a function.

Data Visualization in R

- Most powerful approach to statistical graphs, based on the **Grammar of Graphics**.
- A graphics language, composed of layers, **geoms** (points, lines, regions), each with graphical **aesthetics** (color, size, shape)

```
library(ggplot2)
library(palmerpenguins)
```

An Introduction to ggplot2

R Code

Plot

```
ggplot(data = penguins, aes(x = flipper_length_mm,  
                             y = body_mass_g)) +  
  geom_point(aes(color = species, shape = species))
```

An Introduction to ggplot2

```
ggplot(data = <your data set>, aes(x = <x axis variable>,  
                                     y = <y axis variable>)) +  
  geom_point(aes(size = <size variable>,  
                 color = <color variable>,  
                 shape = <shape variable>))
```

